OdontoAsistente

Chatbot de reservas para un consultorio odontológico

Entrega del segundo corte · Creación de Chatbots con IA (Electiva II) · EAM-2026II

Integrante: Kevin Londoño Hernandez (trabajo individual)

Octubre de 2026

Repositorio: https://github.com/darking2712/Odontoasistente

Este documento se puede leer solo, sin explicación oral. Presenta el negocio, el problema, la solución, cómo funciona el chatbot, las reglas que aplica, las pruebas que lo respaldan y lo que todavía le falta. Todos los datos (consultorio, personas y reservas) son ficticios.

OdontoAsistente en una página

OdontoAsistente es un chatbot que atiende por conversación a quien quiere informarse o pedir una cita en un consultorio odontológico (ficticio) y deja las reservas guardadas, sin que dos citas se crucen.

Qué hace

- Responde sobre servicios, precios y horarios.
- Dice qué horas hay libres para un servicio.
- Crea, consulta, modifica y cancela reservas.
- Siempre pide confirmación antes de guardar.

Cómo lo hace

- La persona escribe en una página de chat.
- Un modelo de lenguaje interpreta el mensaje.
- Un servicio con las reglas del negocio valida la operación.
- Una base de datos conserva las reservas.

Qué demuestra este documento

- Cómo se recorre una solicitud completa.
- Qué reglas evitan reservas cruzadas.
- Qué pruebas se hicieron y qué resultado dieron.
- Qué falló, qué se corrigió y qué falta.

Contenido: 3 negocio y problema · 4 solución y límites · 5 herramientas · 6 funcionamiento · 7–8 recorrido de una solicitud · 9 conversación · 10 reglas de disponibilidad · 11 operaciones · 12–14 pruebas y resultados · 15 evolución frente a Flowise · 16 cierre.

El negocio y el problema

El negocio. Consultorio OdontoAsistente, un consultorio odontológico ficticio en Armenia, Quindío (Calle 45 #12-30, dirección también ficticia). Ofrece cinco servicios:

Servicio	Precio (COP)	Duración
Valoración general	60.000	30 min
Limpieza dental (profilaxis)	90.000	45 min
Blanqueamiento dental	350.000	60 min
Consulta de ortodoncia	70.000	30 min
Extracción simple	120.000	45 min

Horario: lunes a viernes de 8:00 a 18:00; sábado de 8:00 a 13:00; domingo cerrado. Se atiende a **una persona a la vez**.

El problema

- Quien quiere una cita escribe para preguntar precios, servicios y horarios.
- Alguien debe revisar **qué horas están libres**, y no basta mirar la hora de inicio: cada servicio dura distinto (30, 45 o 60 minutos).
- Hay que evitar que dos citas se **crucen**, aunque dos personas pidan la misma hora casi a la vez.
- Después hay que anotar la reserva de forma que se pueda consultar, cambiar o cancelar.
- Hacerlo a mano es lento y propenso a errores, como las dobles reservas.

La solución: qué hace y qué no hace

Usuarios

- Personas que quieren **informarse** sobre el consultorio.
- Personas que quieren **agendar** una cita.
- Personas que ya tienen una cita y quieren **consultarla, cambiarla o cancelarla**.
- Se usa desde una página de chat en el navegador; la persona no instala nada.

Tareas que resuelve

- Mostrar servicios, precios y horarios.
- Consultar las **horas libres** de un servicio en una fecha.
- **Crear** una reserva a partir de una conversación en varios mensajes.
- **Consultar** una reserva con su código.
- **Modificar** servicio, fecha u hora.
- **Cancelar** una reserva y liberar el horario.
- Pedir **confirmación** antes de guardar, modificar o cancelar.

Límites

- No diagnostica, no recomienda tratamientos y no procesa pagos.
- Atiende a una sola persona a la vez (capacidad 1).
- Es una demostración: todos los datos son **ficticios**.
- La API no tiene autenticación: es solo para uso local.
- Si el mensaje es ambiguo, el modelo puede interpretarlo mal; por eso nada se guarda sin confirmar.

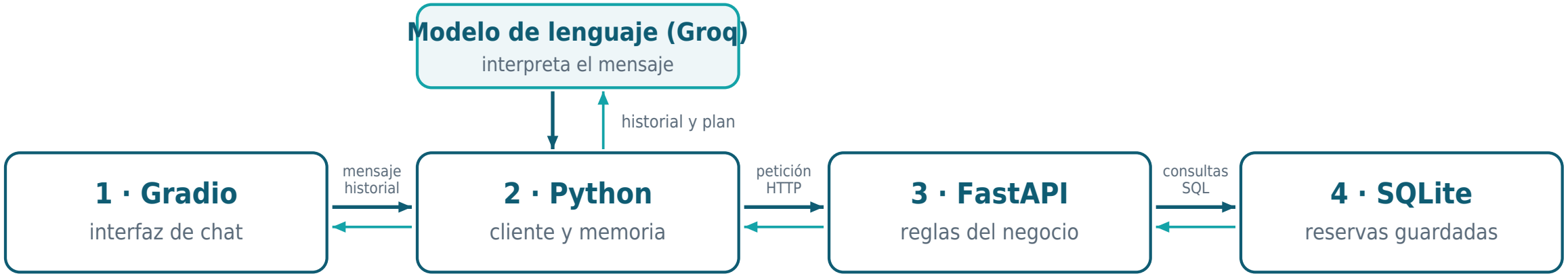
Las herramientas y la tarea de cada una

Herramienta	Qué es	Su tarea en el proyecto
Python	Lenguaje de programación.	Une las piezas: llama al modelo, valida lo que responde y llama a la API.
Modelo de lenguaje (Groq)	Un programa de IA que entiende texto escrito como lo diría una persona. Groq es el servicio que lo ejecuta.	Convertir el mensaje en un **plan**: qué quiere hacer, servicio, fecha, hora, nombre y código.
Gradio	Biblioteca que crea una página web de chat.	Es la pantalla donde conversa la persona y donde pulsa «Confirmar operación».
FastAPI	Biblioteca para crear una **API**: un servicio al que otros programas hacen peticiones por internet (HTTP).	Ejecutar las operaciones del negocio (crear, consultar, modificar, cancelar) y aplicar las reglas.
SQLite	Base de datos que vive en un solo archivo (`reservas.db`).	Conservar las reservas aunque se apague el programa.

Términos que aparecen en el documento

- **Prompt:** las instrucciones que se le dan al modelo para que sepa qué debe devolver.
- **Historial:** la lista de mensajes ya intercambiados; se reenvía al modelo para que «recuerde» la conversación.
- **Sesión:** la conversación de una persona en su navegador; cada ventana tiene la suya.
- **Plan:** el resumen en formato JSON (datos organizados) que devuelve el modelo y que el programa valida.

Cómo funciona: de la conversación a la base de datos



`app_gradio.py`
La persona escribe, ve las respuestas y pulsa **Confirmar**, **Descartar** o **Reiniciar**.

`consulta_llm.py` y `chatbot_memoria.py`
Envía el historial al modelo, recibe el plan, lo **valida** y arma el resumen para confirmar.

`main.py`
Recibe operaciones por HTTP, **aplica las reglas** (horario, duración, cruces) y responde con un código HTTP.

`almacen_sqlite.py` y `reservas.db`
Tabla `reservas`: una fila por reserva. Comprueba el cruce y guarda en una misma transacción.

> **Idea central:** el modelo **solo interpreta**. No guarda reservas, no confirma nada y no conoce precios ni disponibilidad. Eso lo decide el código: la confirmación llega solo cuando la persona pulsa el botón y las reglas las aplica FastAPI antes de escribir en SQLite.

Flechas azules: la petición. Flechas turquesa: la respuesta.

Recorrido de una solicitud (1/2): pedir y revisar el resumen

PANTALLA DE CHAT (Gradio)

Persona: Quiero un blanqueamiento el 20 de octubre a las 10, a nombre de Laura

OdontoAsistente: Voy a crear la reserva: cliente=Laura, servicio=blanqueamiento, fecha=2026-10-20, hora=10:00. ¿Confirmas?

Confirmar operación | **Descartar** | **Reiniciar conversación**

Mientras esa solicitud esté pendiente, el chat no acepta mensajes nuevos: pide confirmar o descartar primero.

Pantalla reconstruida con el texto real que genera el programa. El mensaje de la persona es un ejemplo basado en la reserva de prueba de la clase 08–09.

Qué ocurre por dentro

1. La persona escribe en el chat. **Gradio** entrega el mensaje a Python junto con el historial de la conversación.
2. Python se lo envía al **modelo** y recibe un plan: acción `crear`, cliente, servicio, fecha y hora.
3. Python **valida** el plan: la acción debe ser una de las permitidas y solo se conservan los campos esperados.
4. Si **faltan datos**, el chatbot los pide y la persona los aporta en mensajes siguientes: el historial permite completar la solicitud en varios turnos.
5. Con todos los datos, arma el **resumen** y lo deja pendiente de confirmación. **Todavía no se guardó nada.**

Recorrido de una solicitud (2/2): confirmar y guardar

6. La persona pulsa **Confirmar operación**: es la única forma de autorizar el guardado.
7. Python envía a FastAPI `POST /reservas` con `"confirmado": true`.
8. FastAPI valida los datos y, **en una sola transacción**, comprueba que el intervalo esté libre y guarda la fila en SQLite.
9. La API responde con la reserva y su **código**; Gradio se lo muestra a la persona.

Respuesta de la API (evidencia de la clase 08–09)

```
{
  "codigo": "F45FD1A4",
  "cliente": "Laura",
  "servicio": "blanqueamiento",
  "fecha": "2026-10-20",
  "hora": "10:00",
  "duracion_minutos": 60,
  "estado": "activa",
  "precio_cop": 350000
}
```

Dónde se comprueba: la tabla `reservas` en SQLite

Campo	Valor	Qué significa
`codigo`	`F45FD1A4`	Identifica la reserva: sirve para consultarla, modificarla o cancelarla.
`cliente`	`Laura`	Nombre de la persona.
`servicio`	`blanqueamiento`	Servicio reservado.
`fecha`	`2026-10-20`	Día de la cita.
`hora`	`10:00`	Hora de inicio.
`duracion_minutos`	`60`	Tiempo que ocupa en la agenda; se usa para detectar cruces.
`estado`	`activa`	`activa` o `cancelada`. El precio no se guarda: sale del catálogo.

*Contenido real de `reservas.db` (SELECT * FROM reservas).*

Resultado: la persona pidió el blanqueamiento por chat, confirmó, y la reserva quedó guardada y consultable. Si ese horario hubiera estado ocupado, la API habría respondido 409 y no se habría guardado ninguna fila.

Cómo recuerda la conversación: historial y sesiones

Historial y roles

El modelo no recuerda nada por sí solo: en cada turno el programa le reenvía **todo el historial**. Cada mensaje tiene un rol:

Rol	Contenido
`system`	Las instrucciones (prompt) y la fecha de hoy en Bogotá, para entender «mañana» o «el viernes».
`user`	Lo que escribe la persona.
`assistant`	Un resumen en JSON de lo que se interpretó y lo que respondió el sistema.

- Así, un dato dicho en un mensaje anterior (por ejemplo el nombre) sigue disponible cuando llega la hora.
- Limitación: el programa no junta los datos por su cuenta; el modelo debe reconstruir el plan completo leyendo el historial.

Estado por sesión y botones

Cada ventana del navegador es una **sesión** con su propio estado: el historial, lo que se ve en pantalla y la solicitud pendiente. Se guarda en memoria, no en la base de datos.

Botón	Qué hace
Confirmar operación	Ejecuta la solicitud pendiente: Python llama a la API y el resultado se anota en el historial.
Descartar	Elimina la solicitud pendiente **sin llamar a la API**: no se crea ninguna fila.
Reiniciar conversación	Vacía pantalla e historial (queda solo el mensaje system). Las reservas ya guardadas no cambian.

- Dos ventanas distintas mantienen datos propios; si piden el mismo horario, solo una reserva se acepta.

Reglas del negocio: horario, duración y cruces

- **Horario:** lunes a viernes 08:00–18:00, sábado 08:00–13:00, domingo cerrado.
- **Horas ofrecidas** en pasos de 30 minutos desde la apertura.
- **Duración:** cada servicio ocupa 30, 45 o 60 minutos y la atención debe **terminar** dentro del horario: el sábado la última valoración empieza a las 12:30.
- No se aceptan fechas ni horas que ya pasaron.
- Una persona atendida a la vez: **dos reservas activas no pueden cruzarse**.
- Las reservas canceladas no cuentan: su horario queda libre.

Regla del cruce

> Dos intervalos se cruzan si
> `inicio_nuevo < fin_existente`
> y `fin_nuevo > inicio_existente`
> Si uno empieza justo cuando termina el otro, **no** hay cruce.

Ejemplo del 2026-10-20 (prueba de la clase 08–09)



Resultado real: la disponibilidad de una valoración pasó de «09:30» a «11:00» al reservar el blanqueamiento.

Operaciones sobre las reservas y por qué no se cruzan

Operación	Petición a la API	Qué hace	¿Pide confirmación?
Ver horas libres	`GET /disponibilidad`	Lista las horas libres de un servicio en una fecha.	No
Crear	`POST /reservas`	Comprueba el cruce y guarda una fila nueva con un código de 8 caracteres.	Sí
Consultar	`GET /reservas/{código}`	Devuelve la reserva con su precio.	No
Modificar	`PUT /reservas/{código}`	Cambia servicio, fecha y hora. Puede solaparse con su propio horario anterior.	Sí
Cancelar	`DELETE /reservas/{código}`	Marca la reserva como «cancelada»: libera el horario y **conserva la fila**.	Sí

Por qué no se cruzan aunque lleguen dos peticiones a la vez. Al crear, SQLite abre la transacción con `BEGIN IMMEDIATE`: reserva la escritura **antes** de comprobar. La segunda petición espera, ve el intervalo ocupado y recibe 409. Si algo falla, se deshace todo y no queda una fila a medias.

Códigos de respuesta que ve el sistema: 201 reserva creada · 404 reserva no encontrada · 409 conflicto (cruce o reserva ya cancelada) · 422 datos inválidos (fecha pasada, día cerrado, hora fuera de horario) · 503 la base de datos no respondió.

Prueba manual: rechazo de un intervalo ocupado

Qué se probaba. Que el sistema rechace una reserva que se cruza con otra activa y acepte la que empieza justo cuando termina. **Punto de partida:** Laura tiene un blanqueamiento activo el 2026-10-20 de 10:00 a 11:00.

Solicitud de Pedro	Intervalo	Respuesta de la API	Qué se comprobó en SQLite
Valoración (30 min) a las 10:30	10:30–11:00	**409** «El intervalo solicitado se cruza con una reserva activa»	No se creó ninguna fila de Pedro
Valoración (30 min) a las 11:00	11:00–11:30	**201** reserva aceptada	Aparece la fila de Pedro (código 99C73722)

Contenido de `reservas.db` al final de la prueba

codigo	cliente	servicio	fecha	hora	duracion_minutos	estado
F45FD1A4	Laura	blanqueamiento	2026-10-20	10:00	60	activa
99C73722	Pedro	valoracion	2026-10-20	11:00	30	activa

Qué demuestra. El rechazo viene de la regla de intervalos de la API (no de la interfaz). La solicitud rechazada no deja ninguna fila. Y una reserva que empieza justo al terminar otra es válida: los extremos que se tocan no cruzan.

Qué se observó en la pantalla. Gradio mostró solo «FastAPI respondió con HTTP 409»; el motivo exacto se vio al repetir la petición en `/docs`. Es una limitación pendiente (página 16). Corresponde a las evidencias 7 a 12 de la bitácora.

Pruebas realizadas y resultados

Prueba	Qué debe comprobar	Cómo se probó	Resultado
Conversación en varios turnos	Conserva los datos aportados antes	Manual en Gradio con el modelo real (reserva F45FD1A4) y automática sobre `/chat`	**Cumplida**
Dos sesiones y reinicio	Separa conversaciones y limpia el estado	Manual con ventana normal y privada; automática sobre `/chat`	**Cumplida**
Crear, modificar y cancelar	Cambios correctos, visibles en SQLite	Crear: manual. Modificar y cancelar: automática sobre la API y la base	**Cumplida** en la API y en Gradio
Intervalos ocupados	Rechaza el cruce con una reserva activa	Manual (409 y sin fila nueva) y automática	**Cumplida**
Cancelación y persistencia	Libera horario y conserva filas al reiniciar	Automática (reinicio simulado) y manual	**Cumplida**
Errores	Informa y permite continuar	Clave inválida, cruce, datos inválidos y fallo de SQLite	**Cumplida** con limitaciones

La prueba automática

`probar_demo.py` usa una base SQLite temporal (no toca `reservas.db`) y **no llama al modelo**. Ejecuta **57 comprobaciones** y todas dan [OK]. La interpretación del modelo solo se verificó de forma manual.

Validaciones de la API	Dos peticiones simultáneas	Persistencia al reiniciar	Cruce de intervalos	Modificar	Cancelar	Chat y sesiones	Errores de almacenamiento
11	**2**	**2**	**6**	**8**	**6**	**18**	**4**

Qué falló y cómo se resolvió

Problema	Causa	Solución	Estado
Error HTTP 401 al enviar el primer mensaje	La clave de Groq no estaba cargada en la terminal (causa probable)	Cargar la clave de nuevo y reiniciar Gradio	**Resuelto**
Gradio muestra solo «HTTP 409»	Solo se muestra el código, no el motivo	El motivo se ve en `/docs`; mejora propuesta: mostrar el mensaje de la API	**Abierto**
La prueba automática ya no servía	Seguía usando el archivo JSON anterior	Reescrita para SQLite con base temporal	**Resuelto**
La instalación fallaba	Faltaba Gradio y la versión de FastAPI chocaba con la de Gradio	Versiones compatibles fijadas en `requirements.txt`	**Resuelto**
Código de reserva mal descrito al modelo	El prompt tenía el formato de la versión anterior	Prompt corregido: 8 caracteres	**Resuelto**
Un fallo de SQLite daba error 500	El error no se traducía	Ahora responde 503 con un mensaje claro	**Resuelto**
Cancelar dos veces daba éxito	Comportamiento de la versión JSON	Ahora responde 409: ya está cancelada	Cambio intencional

Evolución: de Flowise y JSON a Python, Gradio y SQLite

2026-09-13 — Flowise + FastAPI + JSON

2026-10-05 — Python llama al modelo, historial y modificar

2026-10-18 — Gradio, SQLite y disponibilidad por intervalos

2026-10-21 — Revisión de cierre, pruebas y documentos

Aspecto	Antes: Flowise + JSON	Ahora: Python + Gradio + SQLite	Qué asumió el código
Interpretar el mensaje	Flujo visual en Flowise con Groq	`interpretar_turno()` en Python	Armar la petición al modelo, extraer y validar el JSON
Memoria	Nodo de memoria de Flowise	Lista de mensajes en el estado de Gradio	Decidir qué se guarda y cuándo se reinicia
Interfaz	Chat de Flowise	Gradio	Botones confirmar, descartar y reiniciar
Confirmación	Palabra CONFIRMAR enviada a `/chat`	Botón «Confirmar operación»	Guardar la solicitud pendiente y ejecutarla solo al confirmar
Guardado	Archivo `reservas.json`	Base SQLite con transacción	Crear la tabla y las consultas; evitar cruces simultáneos
Errores	503 si fallaba el archivo	404, 409, 422 y 503	Traducir cada error a un mensaje comprensible

Con Flowise, la interpretación y la memoria eran piezas visuales ya hechas. Con Python pasaron a ser código propio: hay más control (validación del plan, confirmación fuera del modelo, transacciones) y más responsabilidad (credenciales, errores, historial y pruebas).

Cierre: limitaciones, mejoras y dónde encontrar todo

Limitaciones conocidas

- API sin autenticación: solo para uso local.
- Historial y solicitud pendiente en memoria: se pierden al recargar la página (las reservas, no).
- El modelo debe reconstruir los datos desde el historial y puede interpretar mal un mensaje ambiguo.
- Una persona atendida a la vez; los errores 409 y 422 se ven en Gradio sin su motivo.
- El precio no se guarda: se toma del catálogo actual.
- La prueba automática no cubre al modelo de lenguaje.

Mejoras pendientes

- Mostrar en Gradio el motivo que devuelve la API.
- Aceptar fechas en lenguaje natural (hoy se pide AAAA-MM-DD).
- Recortar el historial en conversaciones largas.
- Agregar autenticación a la API.
- Soportar varios profesionales.
- Incluir una prueba automática de la interpretación del modelo.

Dónde encontrar todo

Repositorio en GitHub
https://github.com/darking2712/Odontoasistente
README: instalación y ejecución
https://github.com/darking2712/Odontoasistente/blob/master/README.md
Bitácora del segundo corte: funcionamiento, decisiones y pruebas
https://github.com/darking2712/Odontoasistente/blob/master/docs/Bitacora_Segundo_Corte.md
Evidencias (capturas y salida de las pruebas)
https://github.com/darking2712/Odontoasistente/tree/master/docs/evidencias